

Projeto - Simulador de Escalonamento de Processos

1. Objetivo

Desenvolver um simulador de escalonamento de processos capaz de gerar cargas de trabalho controladas por seed, executar algoritmos clássicos de escalonamento e comparar os resultados com um algoritmo proposto pela própria equipe.

- Aplicar conceitos de estados de processos, filas de prontos, bloqueio por E/S, preempção e troca de contexto.
- Comparar políticas de escalonamento com base em métricas quantitativas.
- Produzir um relatório em formato de artigo científico, com metodologia, resultados e conclusão.

2. Organização geral

- Equipes com aproximadamente 5 integrantes.
- O código deve ser versionado em Git/GitHub, com commits que indiquem a participação dos integrantes.
- O simulador deve ser implementado em C.
- Scripts auxiliares para geração de gráficos, consolidação de resultados e análise estatística podem ser feitos em outra linguagem, desde que documentados no repositório.

3. Modelo de processo

Cada processo deve ser representado por uma entidade simulada, com atributos suficientes para que os algoritmos de escalonamento possam tomar decisões.

- Identificador do processo.
- Tempo de chegada.
- Prioridade.
 - A equipe deve definir se valores menores ou maiores representam maior prioridade e deve usar essa mesma convenção em todos os experimentos.
- Duração total de CPU ou lista de rajadas de CPU.
- Número de requisições de E/S.
- Duração de cada requisição de E/S.
- Estado atual: novo, pronto, executando, bloqueado ou finalizado.

Uma forma comum de modelar isso é representar cada processo como uma sequência de rajadas:

CPU -> E/S -> CPU -> E/S -> CPU

4. Simulação

- A simulação deve usar tempo discreto ou simulação por eventos.
- A fila de prontos deve conter apenas processos aptos a usar a CPU.
- Quando um processo solicita E/S, ele deve sair da CPU e entrar no estado bloqueado. Ao término da operação de E/S, o processo deve retornar à fila de prontos.
- A equipe deve definir e documentar como as operações de E/S foram modeladas. A descrição deve deixar claro:
 - Quando um processo solicita E/S;
 - Por quanto tempo ele permanece bloqueado
 - Se múltiplas operações de E/S podem ocorrer em paralelo
 - Se existe um ou mais dispositivos de E/S com fila própria
 - Quando o processo retorna à fila de prontos.
- A mesma modelagem de E/S deve ser usada para todos os algoritmos avaliados pela equipe. A equipe deve discutir no relatório como essa escolha de modelagem pode influenciar os resultados.
- O custo de troca de contexto deve ser configurável e maior que zero nos experimentos principais. A equipe deve definir e documentar como a troca de contexto foi modelada, deixando claro:
 - Quando ela ocorre
 - Quanto tempo consome
 - Se durante esse tempo a CPU fica indisponível para execução de processos
 - Se a troca é contabilizada quando a CPU sai do estado ocioso para executar um processo
- O valor do custo de troca de contexto deve ser o mesmo para todos os algoritmos avaliados em um mesmo experimento.
- A equipe pode realizar experimentos adicionais com custo de troca de contexto igual a zero, desde que isso seja apresentado como análise complementar.

5. Geração das cargas de trabalho

As características dos processos devem ser geradas aleatoriamente a partir de uma seed. A mesma seed, no mesmo cenário, deve gerar exatamente a mesma carga de trabalho.

Configuração mínima para os experimentos principais:

- Pelo menos 1.000 processos por execução
- Pelo menos 100 seeds por cenário
- Mesmos cenários e mesmas seeds para todos os algoritmos

Configuração ampliada opcional:

- 10.000 ou mais processos por execução
- Mais de 100 seeds por cenário
- Cenários adicionais ou análises de sensibilidade.

Observações:

- A configuração ampliada não substitui a configuração mínima. Ela pode ser usada para aprofundar a análise, desde que os resultados sejam discutidos no relatório.
- Os tempos de chegada dos processos também devem ser gerados a partir da seed. A equipe deve documentar se os processos chegam todos no instante 0, em intervalos fixos, em intervalos aleatórios ou em lotes.
- O modelo de chegada deve ser o mesmo para todos os algoritmos avaliados dentro de um mesmo cenário.
- A chegada de todos os processos no instante 0 é permitida apenas se a equipe justificar essa escolha e discutir sua limitação.

6. Cenários obrigatórios

1. Cenário aleatório equilibrado: mistura de processos curtos, longos, com pouca e muita E/S.
2. Cenário I/O-bound: processos com rajadas de CPU menores e mais requisições de E/S.
3. Cenário CPU-bound/processos longos: processos com rajadas de CPU maiores e menos requisições de E/S.
4. Cenário com prioridades desbalanceadas: muitos processos com prioridade alta e alguns com prioridade baixa

7. Seeds e rigor estatístico

- Mínimo obrigatório: 100 seeds diferentes por cenário.
- Meta recomendada: 1000 seeds por cenário, se o tempo de execução permitir.
- Para cada métrica, apresentar média e Intervalo de Confiança (IC) de 95%.
- Os gráficos devem mostrar a média e o IC (sombreado, barras ou de outra forma)

Fórmula sugerida para IC95%:

$$IC_{95\%} = \bar{x} \pm 1,96 \cdot \frac{s}{\sqrt{n}}$$

Onde:

- $\bar{x}$: média amostral dos resultados obtidos nas execuções;
- s : desvio padrão amostral dos resultados;
- n : número de execuções independentes realizadas, por exemplo, número de seeds utilizadas.

8. Algoritmos de escalonamento

Algoritmos clássicos obrigatórios

- FCFS - First-Come, First-Served.
- Round Robin, com quantum configurável.
- Escalonamento por Prioridade não preemptivo
- Algoritmos adicionais podem ser implementados como comparação extra, desde que a equipe explique quais informações o algoritmo usa e se alguma delas representa conhecimento futuro.

Algoritmo próprio

- Cada equipe deve propor e implementar um algoritmo de escalonamento com contribuição própria. O algoritmo não precisa ser inédito na literatura, mas deve apresentar uma diferença clara em relação aos algoritmos clássicos obrigatórios e em relação a qualquer algoritmo usado como inspiração.
- O algoritmo pode ser:
 - Uma criação original da equipe
 - Uma adaptação de um algoritmo conhecido
 - Uma combinação de estratégias existentes
- Caso a equipe utilize uma ideia encontrada em artigos, livros, sites, vídeos, códigos ou outros materiais, a fonte deve ser citada no relatório. Porém, não será aceito apenas copiar, renomear ou reimplementar sem alterações um algoritmo já existente.
- Para todos os algoritmos próprios, a equipe deve explicar:
 - Qual problema o algoritmo tenta resolver
 - Quais informações o algoritmo usa para tomar decisões
 - Como o próximo processo é escolhido
 - Por que o algoritmo deveria melhorar algum aspecto da simulação
 - Quais limitações ele possui.
- Se o algoritmo for baseado em uma ideia conhecida, a equipe também deve explicar:
 - Qual algoritmo ou estratégia serviu de base;
 - O que foi modificado
 - Por que a modificação foi feita
- O algoritmo não deve usar informações futuras que um sistema operacional real não teria no momento do escalonamento. Ele deve tomar decisões apenas com base no estado atual da simulação e no histórico já observado.

9. Resultados esperados

Métricas Obrigatórias

As equipes devem avaliar os algoritmos usando, no mínimo, as seguintes métricas:

Métrica	O que mede
Turnaround médio	Tempo médio entre a chegada e o término dos processos. Mede o desempenho geral do algoritmo.
Trocas de contexto	Quantidade de mudanças de processo executando na CPU. Mede o comportamento e o custo potencial do escalonador.
Jain do slowdown	Grau de equilíbrio entre os slowdowns dos processos. Usado como indicador de justiça.

As métricas obrigatórias principais são: turnaround médio, trocas de contexto e índice de Jain aplicado ao slowdown. O slowdown é uma métrica auxiliar usada para calcular o índice de justiça.

Slowdown

O slowdown de um processo mede quanto tempo ele levou para terminar em relação ao menor tempo possível para sua execução:

$$slowdown_i = \frac{turnaround_i}{tempo\ mínimo\ ideal_i}$$

O tempo mínimo ideal corresponde ao tempo que o processo levaria para terminar se não esperasse na fila de prontos. Em um modelo com rajadas, pode ser calculado como:

$$tempo\ mínimo\ ideal_i = \sum CPU_i + \sum E/S_i$$

Justiça com índice de Jain

A justiça deve ser calculada usando o índice de Jain aplicado aos slowdowns dos processos:

$$Jain_{slowdown}(\%) = \frac{(\sum_{i=1}^n slowdown_i)^2}{n \cdot \sum_{i=1}^n slowdown_i^2} \times 100$$

onde n é o número de processos simulados.

Valores próximos de 100% indicam que os processos tiveram slowdowns semelhantes. Valores menores indicam maior desigualdade entre os processos, ou seja, alguns processos foram proporcionalmente mais prejudicados que outros.

Obs:A justiça deve ser interpretada junto com o turnaround médio. Um algoritmo pode ser “justo” por prejudicar todos os processos de forma parecida, mas ainda assim ter desempenho ruim.

10. Gráficos e comparações

- A equipe deve comparar o algoritmo próprio com todos os algoritmos clássicos obrigatórios, em todos os cenários obrigatórios.
- Os resultados devem usar gráficos com média e IC95% para as métricas principais.
- A discussão deve ser direcionada principalmente ao algoritmo próprio, respondendo:
 - Em quais cenários ele teve melhor desempenho
 - Em quais métricas ele melhorou
 - Em quais cenários ele teve pior desempenho
- Quais custos ele introduziu, como mais trocas de contexto ou maior turnaround
- Por que esses resultados aconteceram?
- Evite conclusões vagas como “o algoritmo X é melhor”. A conclusão deve estar ligada ao cenário, à métrica analisada e à comparação com os algoritmos clássicos.
- Todos os gráficos devem identificar claramente a métrica, os cenários, os algoritmos e a unidade de medida utilizada.

11. Relatório em formato de artigo

Os algoritmos clássicos obrigatórios devem ser descritos de forma resumida, apenas o suficiente para tornar a comparação reproduzível. A equipe deve informar as escolhas de implementação relevantes, como critérios de desempate, quantum do Round Robin, interpretação da prioridade.

A descrição detalhada deve ser dedicada ao algoritmo próprio, incluindo motivação, funcionamento, informações usadas para tomar decisões, diferenças em relação aos algoritmos clássicos e limitações.

O artigo deve ter de 4 a 6 páginas e conter:

- Título, autores e resumo.
- Introdução e motivação do estudo.
- Descrição do modelo de sistema proposto.
- Descrição dos algoritmos implementados, incluindo o algoritmo próprio.
- Metodologia experimental: cenários, seeds, número de processos e métricas.
- Resultados com tabelas e gráficos.
- Discussão dos resultados.
- Conclusão.
- Referências, quando utilizadas.

Links com Templates:

- Word ou Latex - <https://www.ieee.org/conferences/publishing/templates>
- Latex - [Link Overleaf](#)
- Word - [Link Word](#)

12. Entregáveis

- Link do repositório Git/GitHub.
- Relatório em formato de artigo, em PDF.
- Slides da apresentação.
- Lista de integrantes e responsabilidades.

13. Apresentação

- Cada equipe deve apresentar a motivação, o modelo de simulação, o algoritmo próprio e os principais resultados.
- A apresentação deve defender as decisões tomadas e explicar os gráficos.
- A avaliação deve considerar se a equipe sabe interpretar os resultados, não apenas mostrar que o programa executa.
- É obrigatória a presença na apresentação das demais equipes. A falta de presença resultará em redução da pontuação individual do estudante.
- Todos os integrantes devem participar da apresentação, demonstrando domínio sobre alguma parte do trabalho.
- Duração de 10 a 12 minutos
- Apresentações que excederem o tempo poderão ser interrompidas.
- A ordem de apresentação será definida pelo professor no dia da apresentação

14. Critérios de avaliação

Critério	Pontos	O que será observado
Artigo em formato científico	6,0	Clareza do problema, modelo de simulação, algoritmo próprio, metodologia, gráficos com IC95%, discussão crítica e conclusão ligada aos resultados.
Apresentação e defesa	3,0	Domínio do projeto, explicação do algoritmo próprio, interpretação dos gráficos, clareza na comparação com os algoritmos de referência, tempo.
Reprodutibilidade e organização do repositório	1,0	Repositório organizado, README, seeds, comandos/scripts de execução e resultados consolidados utilizados no artigo.

15. Datas Importantes

Dia	Detalhes
19/08/2026	Entrega do trabalho e primeiro dia de apresentações
21/08/2026	Segundo dia de apresentações

A entrega do trabalho deve ser realizada até o dia 19/08/2026, independentemente da data de apresentação da equipe.